CIFAR-10 Image Classification

Project report and interview guide: a CNN built from scratch in PyTorch, a residual-network improvement, and an honest comparison

The one-sentence answer
I built a complete CIFAR-10 pipeline in PyTorch (data, model, training, evaluation, comparison, ONNX export, API, tests). The baseline CNN reached **81.70%** test accuracy; my residual-network variant reached **75.32%**, so it did **not** improve - and I reported that honestly instead of tuning on the test set.

Numbers to remember (all taken from the real result files)

Item	Baseline CNN	ResNet CNN
Parameters	288,746	308,650
Test accuracy (10,000 images)	81.70%	75.32%
Test loss	0.5302	0.6979
Best validation accuracy (5,000 images)	82.92%	75.70%
Macro F1 on test	0.8156	0.7518
Training time on CPU (10 epochs)	21.7 min	81.2 min
Weakest classes (test accuracy)	cat 62.9%, bird 68.8%, dog 76.8%	cat 57.0%, bird 58.5%, dog 66.1%
Strongest classes	automobile 92.0%, ship 91.8%, truck 90.7%	ship 91.4%, automobile 88.8%, truck 88.3%

- **Data:** 45,000 train / 5,000 validation / 10,000 test. Normalised per channel. Augmentation: random crop (padding 4) and horizontal flip, train only.
- **Training:** SGD (momentum 0.9, weight decay 5e-4), OneCycleLR with peak 0.1, batch 128, 10 epochs, seed 42, no gradient clipping, no mixed precision.
- **Hardware:** CPU only (Snapdragon X, 8 cores, Windows 11 ARM64), PyTorch 2.14.1. No GPU was used.
- **Gap:** ResNet is 6.38 percentage points below the baseline; a test accuracy near 82% has a standard error of about 0.4 pp, so this is not noise.
- **Code quality:** 57 unit tests pass in about 9 seconds; everything is configured from one YAML file; the repo is on GitHub.
- **Repository:** github.com/nivitsm2011/everquint-ai-engineer-assessment, folder *cifar10-classification*.

1. Have we met all the requirements?

Yes. Every requirement in the assignment is implemented and was verified by running it. Two points need honest wording in an interview: the Part 2 improvement did **not** beat the baseline (the requirement is to implement it and compare, which is done), and mixed precision is wired in but could not be run on a CUDA GPU because this machine has none.

Requirement	Where it is fulfilled	Status
Data pipeline: load, normalisation, augmentation, batching, DataLoader	src/data.py: get_datasets, build_transforms, split_indices, get_dataloaders	Done
CNN from scratch with Conv, BatchNorm, Dropout, classification head	src/models.py: ConvBlock, BaselineCNN (no pretrained weights)	Done
Training loop: loss, accuracy, LR scheduling	src/engine.py: train_one_epoch, evaluate, build_scheduler; train.py	Done
Checkpoints (best and last, resume)	src/engine.py: save_checkpoint, load_checkpoint; train.py --resume	Done
Loss and accuracy curves	src/visualize.py: plot_training_curves	Done
Test accuracy on the untouched test set	evaluate.py (baseline 81.70%)	Done
Sample misclassified images	evaluate.py: select_misclassified; visualize.py: plot_misclassified	Done
Per-class metrics and confusion matrix	evaluate.py: compute_metrics; visualize.py: plot_confusion_matrix	Done
Random seeds / reproducibility	src/utils.py: set_seed, seed_worker, make_generator	Done
Requirements file	requirements.txt (15 pinned packages)	Done
README: structure, training setup, design decisions, how to run	README.md	Done
Part 2: residual block improvement	src/models.py: ResidualBlock (projection shortcut), ResNetCNN	Done
Baseline vs improved comparison	compare.py -> outputs/results/comparison.md and comparison plot	Done (improvement did not help)
Optional: mixed precision	train.py --amp, engine.make_grad_scaler	Done on CPU; CUDA path untested
Optional: ONNX export + inference check	export_onnx.py (max logit difference 5.7e-06, 100% agreement)	Done
Optional: FastAPI deployment	app/main.py: /health, /predict	Done
Optional: unit tests	tests/ (57 tests)	Done
Optional: DDP / TPU	Not implemented; noted in README as out of scope	Out of scope (as agreed)

2. How to talk about the project

The 30-second version

Say it like this:
I built an image classifier for CIFAR-10 in PyTorch, completely from scratch - no pretrained models. A small three-block CNN with BatchNorm and Dropout reaches 81.7% on the test set. Then I tried the assignment's improvement, residual blocks, under identical settings. It scored lower, 75.3%, and I report that honestly. I also exported the model to ONNX, served it with FastAPI, and covered the code with unit tests.

The 2-minute walkthrough

- **Problem and constraints.** Classify 32x32 colour images into 10 classes. I had about four hours and only a CPU, so I kept the model small (about 0.29 million parameters) and trained for 10 epochs (about 22 minutes).
- **Data.** I split the 50,000 training images into 45,000 train and 5,000 validation with a seeded shuffle and kept the official 10,000 test images untouched until the end. Normalisation uses statistics computed on the training split only. Augmentation (random crop, flip) is applied to training images only.
- **Model.** Three blocks of two 3x3 convolutions with BatchNorm and ReLU, then max-pooling and light spatial dropout; channels 32, 64, 128; global average pooling; dropout; one linear layer. Kaiming initialisation.
- **Training.** SGD with momentum and weight decay, a OneCycle learning-rate schedule, best checkpoint chosen on validation accuracy, early stopping, resumable checkpoints.
- **Results.** 81.70% test accuracy. The model confuses cats with dogs most, which is what you would expect at 32x32 resolution. Neither train nor validation curve shows overfitting; the model is still improving at epoch 10.
- **Improvement.** I added residual blocks with projection shortcuts, matched in size (308,650 vs 288,746 parameters). Same seed, data, optimiser and schedule. It reached 75.32% and trained 3.7 times slower, so the residual variant did not help in this setup.
- **Engineering.** One YAML config, command-line overrides, seeded and reproducible, ONNX export verified against PyTorch, a FastAPI endpoint, and 57 unit tests.

Why a negative result is still a good story

Say it like this:
The comparison was fair - same seed, data, optimiser, schedule and epochs, with comparable parameter counts - so the result is trustworthy even though it is not the one I hoped for. I did not tune on the test set to rescue it. I can list what I would test next: a longer schedule, max-pooling instead of strided convolutions in the ResNet, and several seeds.

Be ready for this
Be able to explain every file and every number in this document in your own words, and answer truthfully if asked how the project was built and which tools you used. Interviewers usually probe the reasoning behind choices, not just the results.

3. What was built, phase by phase

The project was developed in eight phases. Each phase was run and checked before the next one started.

Phase	What was done	How it was checked
1. Setup and reproducibility	Project structure; YAML config loaded into a validated dataclass with CLI overrides; seeding utilities (Python, NumPy, PyTorch, CUDA, DataLoader workers); logger, timer, average meter; device auto-detection (CUDA, MPS, CPU); pinned requirements; virtual environment.	Same seed gave identical random tensors; config validation rejected bad values; device printed as CPU.
2. Data pipeline	Downloaded CIFAR-10; seeded 45k/5k split; per-channel normalisation computed from the training split; augmentation for train only; DataLoaders with workers and a seeded generator; denormalise helper; sample-image grid.	Batch shape [128, 3, 32, 32]; labels 0-9; no train/val overlap; validation images not augmented; one train epoch took 9-11 s with 4 workers.
3. Baseline model	ConvBlock and BaselineCNN (32-64-128 channels), model factory, parameter counter.	Output [8, 10]; 288,746 parameters; first-layer weight spread matched the Kaiming expectation.
4. Training engine	Training and evaluation loops, optimiser and scheduler factories, checkpoints (best and last), resume, early stopping, AMP and gradient-clipping hooks, train.py CLI.	Quick smoke run: loss fell and the LR followed the OneCycle shape; resumed run matched an uninterrupted run's LR schedule; full epoch measured at about 166 s.
5. Train and evaluate baseline	Trained 10 epochs; wrote the evaluation script and plots: curves, confusion matrix, per-class report, misclassified grid, metrics JSON.	Test accuracy 81.70%; all plots opened and checked.
6. Residual improvement	ResidualBlock with projection shortcut, ResNetCNN, trained with identical settings, comparison script and report.	Test accuracy 75.32%; report numbers cross-checked against the metrics files.
7. Optional extensions	--amp flag; ONNX export with dynamic batch; FastAPI service; unit tests.	ONNX matched PyTorch on 100 images; API tested with real test images and a live server; all tests passed.
8. Documentation and audit	Full README, requirements audit, clean-up (unused imports and dependency removed), extra tests, git history, push to GitHub.	Fresh smoke run and 57 tests passed; remote branch matches local.

4. Why each technique was chosen

For each choice: what it is, why it was used, and a one-line answer you can say out loud. None of these choices was ablated (tested against alternatives) except where stated, so describe them as reasoned design decisions, not proven optima.

Train / validation / test split

What: 50,000 official training images split 45,000 / 5,000 with a seeded shuffle; the 10,000 test images are used once, at the end.

Why: The validation set is used for every decision (best checkpoint, early stopping). If the test set were used for decisions, the final accuracy would be optimistic. Two separate dataset objects share the same indices so validation images never get training augmentation.

> **Say it like this:**
> I keep the test set untouched until the end, and choose checkpoints on a separate validation set, so the reported number is an honest estimate.

Per-channel normalisation

What: Subtract the channel mean and divide by the channel standard deviation: mean (0.4913, 0.4822, 0.4466), std (0.2470, 0.2436, 0.2617).

Why: Inputs centred near zero with unit scale make optimisation faster and more stable. The statistics come from the training split only, so no information from validation or test leaks in. They match the commonly quoted full-train values to within 0.0001.

> **Say it like this:**
> Normalising inputs makes gradient descent better conditioned, and computing the statistics on the training split only avoids data leakage.

Data augmentation: random crop with padding 4, horizontal flip

What: Pad the image by 4 pixels and crop a random 32x32 window; flip left-right with probability 0.5; training images only.

Why: A cat is still a cat when shifted a few pixels or mirrored, so these transformations create new valid examples and act as a regulariser. I kept to these standard two on purpose. Validation and test images get no augmentation, so they measure true performance.

> **Say it like this:**
> Crop and flip encode invariances I know are true for these classes, which reduces overfitting for free.

Convolutional network

What: Stacked 3x3 convolutions that share weights across the image.

Why: Convolutions exploit the local structure of images and need far fewer parameters than a fully connected network on 3,072 inputs. Stacking 3x3 layers builds a large receptive field from small kernels.

> **Say it like this:**
> Convolution weights are shared across positions, so the network learns local features such as edges and textures with few parameters.

Depth and width: three blocks, two convolutions each, 32-64-128 channels

What: Resolution goes 32x32, 16x16, 8x8; channels double after each max-pool. 288,746 parameters.

Why: A standard small VGG-style layout. Doubling channels as resolution halves keeps the compute per stage roughly balanced. The widths are small because training ran on a CPU in limited time; a 64-128-256 version

would have about four times the parameters and compute.

> **Say it like this:**
> I picked a small VGG-style network so it trains in about 22 minutes on a CPU. I did not run an architecture search, so I describe it as a sensible default, not an optimum.

Batch normalisation after each convolution

What: Normalises each channel using batch statistics during training and running averages at evaluation.

Why: Stabilises training, allows a high learning rate (peak 0.1), and adds mild regularisation. Because a BatchNorm layer follows each convolution, the convolutions have no bias (the BatchNorm shift replaces it).

> **Say it like this:**
> BatchNorm keeps activations well-scaled so I can train with a high learning rate; that is why the convolutions have no bias.

Dropout: Dropout2d(0.1) after each block, Dropout(0.3) before the classifier

What: Randomly zeroes whole feature maps (spatial dropout) inside the network and individual units before the final layer.

Why: Neighbouring pixels in a feature map are correlated, so dropping single pixels barely hurts; dropping whole channels is a stronger, more useful regulariser. The head uses ordinary dropout at 0.3 because it works on a small dense vector.

> **Say it like this:**
> Spatial dropout in the conv blocks, standard dropout in the head - each type matched to where it is applied.

Global average pooling instead of flatten + dense layers

What: Averages each of the 128 final feature maps to one number, then one linear layer maps 128 values to 10 logits.

Why: The head has only 128 x 10 + 10 = 1,290 parameters, which limits overfitting and model size. It also makes the network independent of the exact input size.

> **Say it like this:**
> Global average pooling replaces a big dense layer, so the classifier is tiny and overfits less.

Kaiming (He) initialisation

What: Convolution weights drawn from a normal distribution with variance scaled for ReLU (fan-out mode).

Why: Keeps the signal and gradient magnitudes stable through ReLU layers at the start of training. The measured first-layer weight standard deviation was 0.0841 against the expected 0.0833.

> **Say it like this:**
> Kaiming initialisation is designed for ReLU networks; it prevents activations from shrinking or exploding at the start.

Cross-entropy loss on raw logits

What: The model outputs logits; softmax is applied inside the loss function.

Why: Standard for multi-class classification, numerically stable, and a freshly initialised model should start near ln(10) = 2.30, which I used as a sanity check.

> **Say it like this:**
> I output raw logits and use cross-entropy, which combines softmax and log-likelihood in a numerically stable way.

SGD with momentum 0.9 and weight decay 5e-4

What: Optimiser for all weights; AdamW is also implemented and selectable in the config.

Why: SGD with momentum is a strong, well-understood choice for convolutional image classifiers and often generalises better than adaptive optimisers. Weight decay is L2 regularisation that discourages large weights.

> **Say it like this:**
> SGD with momentum is the classic choice for CNNs on CIFAR; weight decay adds regularisation. The config also supports AdamW.

OneCycle learning-rate schedule

What: Learning rate rises from 0.004 to a peak of 0.1 (reached around epoch 4), then anneals towards zero; stepped every batch.

Why: With only 10 epochs, a warm-up followed by a long decay lets the model train fast at a high LR early and settle into a good minimum at the end. The per-epoch values were 0.004, 0.028, 0.076, 0.100, 0.095, 0.081, 0.061, 0.039, 0.019, 0.005.

> **Say it like this:**
> OneCycle gives a short warm-up and a long anneal, which suits a short training budget. A cosine schedule is also available.

Checkpointing and early stopping

What: last.pt every epoch; best.pt when validation accuracy improves; early stopping with patience 5 on validation accuracy.

Why: Saving the best model protects against a late bad epoch, and last.pt lets training resume after an interruption. Early stopping never triggered here because validation accuracy rose every epoch.

> **Say it like this:**
> I keep the best model by validation accuracy and can resume from the last checkpoint; early stopping is there but did not trigger.

Gradient clipping and mixed precision (implemented, off by default)

What: Optional gradient-norm clipping; optional autocast with a gradient scaler, active only on CUDA.

Why: Clipping guards against exploding gradients and AMP speeds up GPU training. Both are hooks that stay off unless requested, so the baseline behaves exactly as described. The CUDA path could not be tested without a GPU; the scaler logic was exercised on CPU in a unit test.

> **Say it like this:**
> They are implemented and switchable, but I only claim what I could test: the CUDA path is untested because I had no GPU.

Residual block with projection shortcut (Part 2, Option A)

What: Output = ReLU(F(x) + shortcut(x)), where F is Conv-BN-ReLU-Conv-BN. If the stride or channel count changes, the shortcut is a 1x1 convolution plus BatchNorm.

Why: Skip connections give gradients a direct path and let a block learn a correction to its input, which is what allows very deep networks to train. The projection is needed so the two branches have the same shape and can be added. Widths match the baseline (32-64-128) so the comparison is about the architecture, not capacity.

Say it like this:
A residual block adds the input to the block's output; a 1x1 projection aligns shapes when they change. I matched model size so the comparison is fair.

ONNX export and FastAPI service

What: The model is exported to a single ONNX file with a dynamic batch axis and served with FastAPI; the model is loaded once at startup.

Why: ONNX decouples inference from PyTorch and runs on onnxruntime; I verified it against PyTorch on 100 images (max logit difference 5.7e-06, 100% same predictions). FastAPI gives typed request handling and automatic docs; invalid uploads return 400, oversized files 413, missing files 422.

Say it like this:
I export to ONNX so inference does not need PyTorch, and verify numerical agreement. The API loads the model once and validates inputs.

5. Results

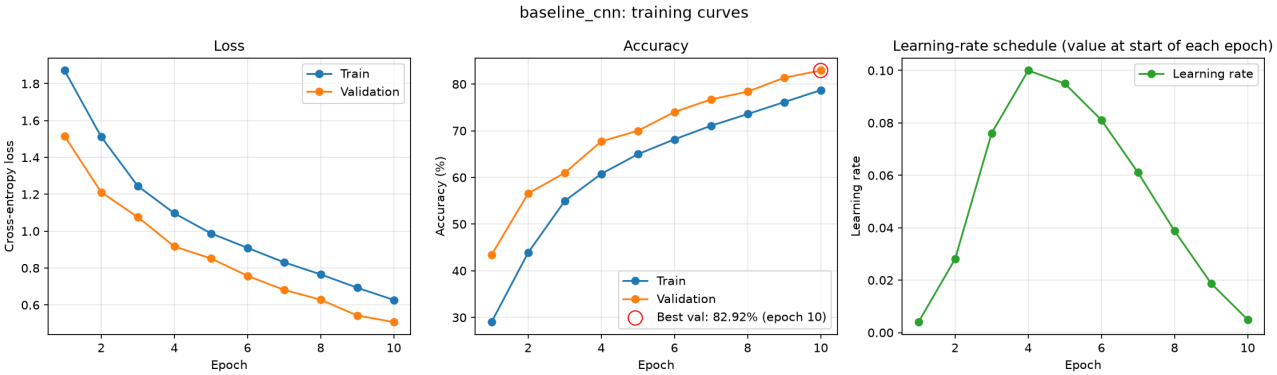
Baseline CNN versus residual CNN

Metric	Baseline CNN	ResNet CNN	Difference
Parameters	288,746	308,650	+19,904
Training time (min)	21.7	81.2	+59.5
Best validation accuracy	82.92%	75.70%	-7.22 pp
Final train accuracy	78.70%	71.20%	-7.50 pp
Test accuracy	**81.70%**	**75.32%**	**-6.38 pp**
Test loss	0.5302	0.6979	+0.1677
Macro F1 (test)	0.8156	0.7518	-0.0639

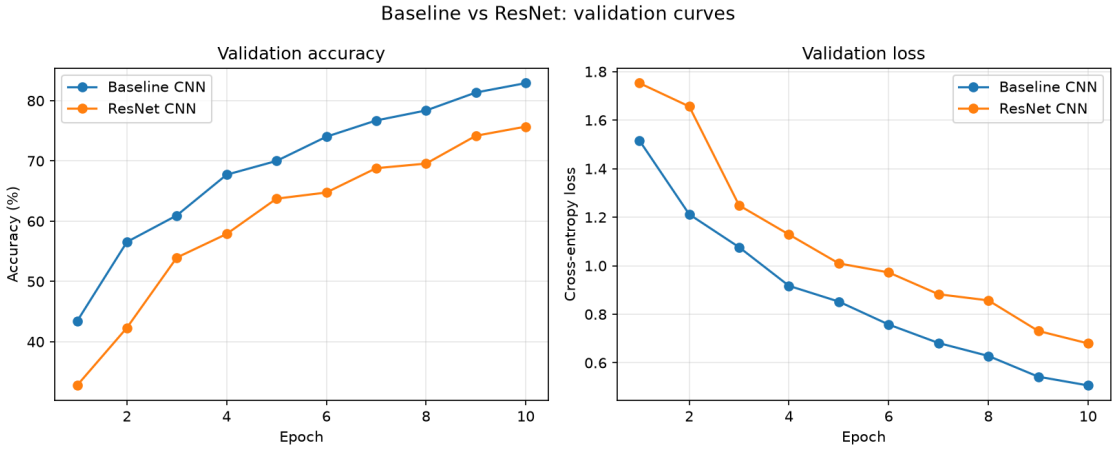
Per-class test accuracy

Class	Baseline	ResNet	Difference
airplane	83.6%	75.6%	-8.0 pp
automobile	92.0%	88.8%	-3.2 pp
bird	68.8%	58.5%	-10.3 pp
cat	62.9%	57.0%	-5.9 pp
deer	79.6%	69.0%	-10.6 pp
dog	76.8%	66.1%	-10.7 pp
frog	87.5%	83.3%	-4.2 pp
horse	83.3%	75.2%	-8.1 pp
ship	91.8%	91.4%	-0.4 pp
truck	90.7%	88.3%	-2.4 pp

Training curves



Baseline CNN: loss, accuracy and learning-rate schedule. The red circle marks the best validation epoch (epoch 10).

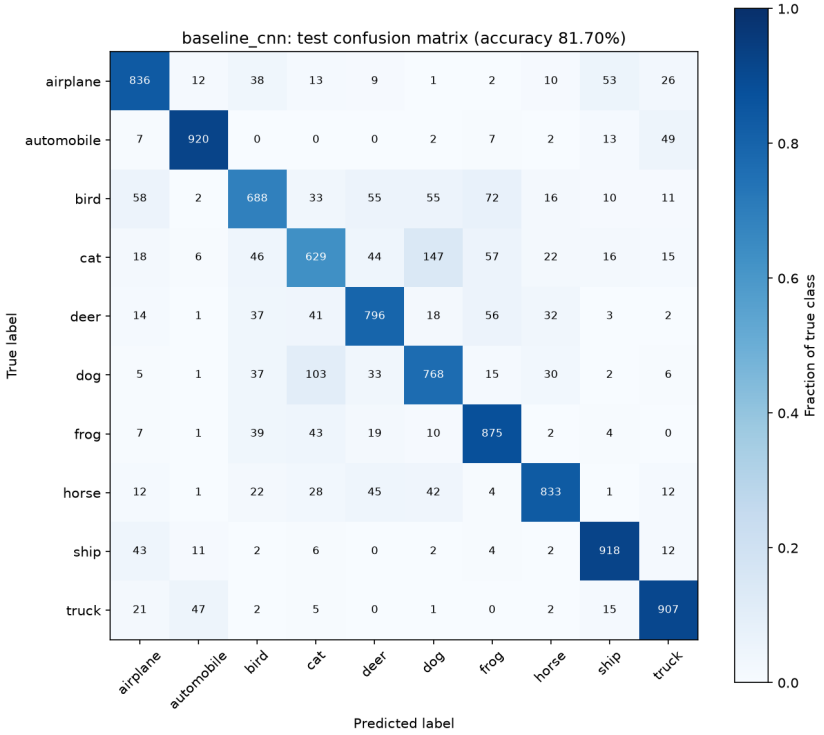


Validation accuracy and loss for both models. The ResNet is behind at every epoch.

How to read the curves

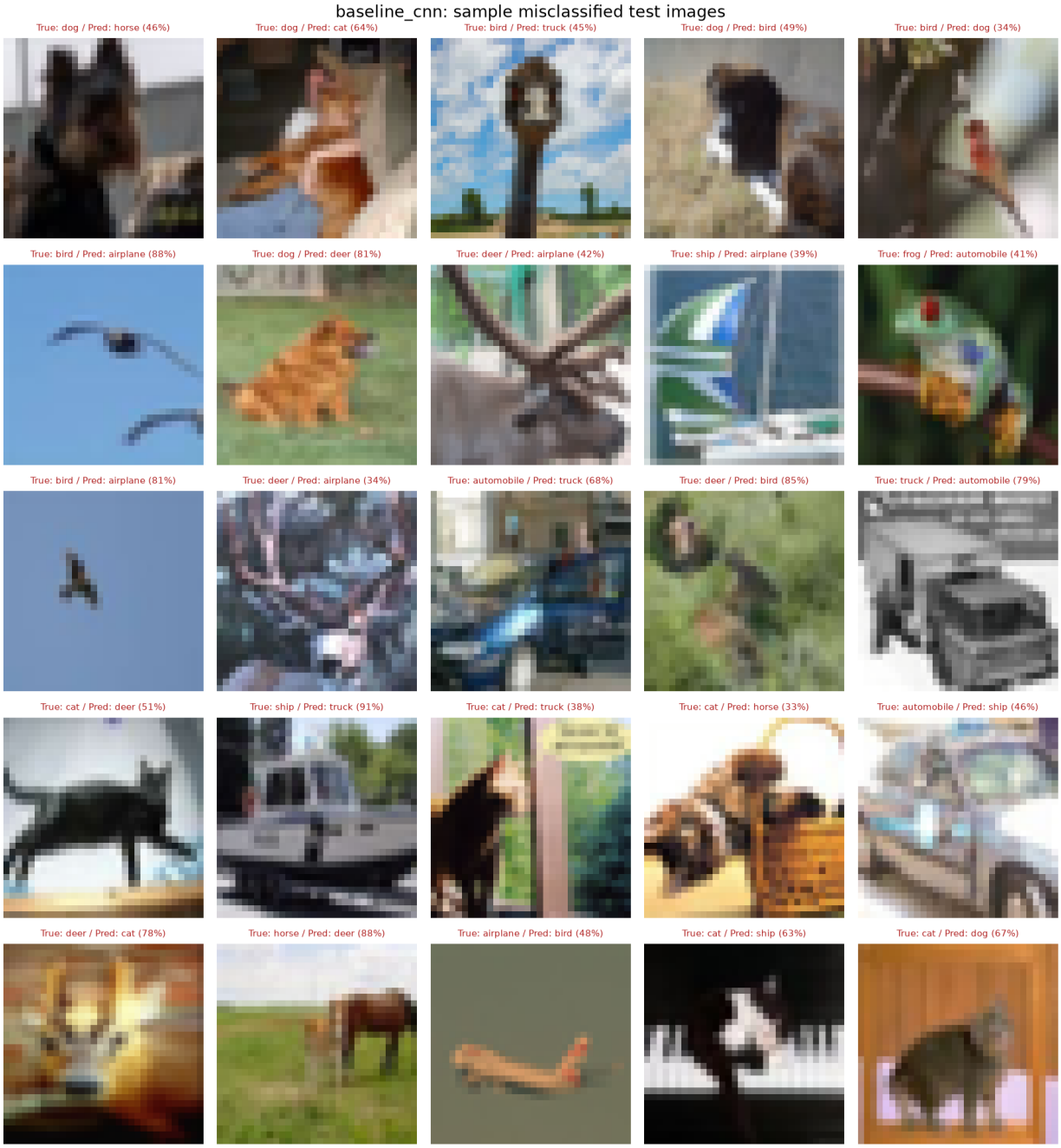
- **No overfitting:** validation accuracy is above training accuracy at every epoch. The training numbers are averaged over the epoch while the weights are still improving, and are measured with dropout and augmentation switched on; validation is measured once, with dropout off.
- **Under-training:** both losses are still falling at epoch 10, so a longer schedule would likely help both models.
- **ResNet slower to learn:** after epoch 5 its validation accuracy was 63.74% against 70.00% for the baseline.
- **Learning-rate panel:** the OneCycle shape - climb to 0.1, then anneal - is visible and explains why accuracy improves fastest in the last epochs.

Confusion matrix and misclassified images (baseline)



Baseline confusion matrix on the 10,000 test images: colour is the fraction of each true class, numbers are counts.

Misclassified test images (baseline)



Random sample of 25 misclassified test images with true class, predicted class and confidence.

Which classes are confused, and why

Baseline: true -> predicted	Count	ResNet: true -> predicted	Count
cat -> dog	147	dog -> cat	179
dog -> cat	103	cat -> dog	171
bird -> frog	72	bird -> deer	103
bird -> airplane	58	bird -> frog	94
cat -> frog	57	airplane -> ship	90
deer -> frog	56	deer -> frog	83

- **Cat and dog** are the main confusion for both models (250 baseline errors between them). They share body shape, fur texture and often indoor backgrounds, and at 32x32 the distinguishing facial detail is only a few pixels wide.
- **Bird** (68.8%) is mistaken for frog, airplane and deer. In the sampled errors, two small birds against plain sky were called airplane with 81% and 88% confidence: small objects and strong background cues push the model to the background class.
- **Vehicles** are easiest (automobile 92.0%, ship 91.8%, truck 90.7%); their remaining errors are between related classes - automobile and truck (49 and 47), airplane and ship (53 and 43).
- **Caveat to say out loud:** these are readings of the confusion matrix and sampled images, not tested hypotheses.

6. Mistakes and problems faced

Interviewers like to hear how you handle things going wrong. Each item below is real: what happened, how it was fixed, and what to say.

Problem	What happened	Fix and lesson
Virtual environment would not build	Creating the venv inside the project failed (ensurepip error) because the project path was very long, hitting a Windows path-length limit.	Created the venv at a short path outside the project; added a note to the README. Lesson: check path length on Windows early.
Dataset download hung	torchvision's download sat at 0 bytes for over 10 minutes, although the host was reachable.	Downloaded the archive with curl, which can resume (about 170 KB/s, about 15 minutes); torchvision then verified and extracted it. Lesson: diagnose with a direct request before waiting longer.
Learning rate read as a string	YAML 1.1 parses 1e-3 as text, so --learning_rate 1e-3 reached the code as a string.	Caught by the Phase 1 check; config loading now converts numeric fields to their declared types.
Gradient clipping on by default	The first config enabled clipping at 1.0, but the Phase 4 brief said hooks should be off by default.	Changed the default to 0 (off) and said so in the report.
Missing epoch count	The Phase 5 message still contained the placeholder text [ENTER NUMBER].	Asked instead of guessing; 10 epochs was chosen. Lesson: never start a long run on an assumption.
Wrong time estimate for the ResNet	I predicted 30-40 minutes from a speed benchmark (1.3x the baseline per batch); the run took 81 minutes (3.7x). Epoch time also drifted from about 330 s early to about 520 s later.	Reported the overrun openly. About 40 extra minutes of the budget were spent; the cause (CPU load or throttling, memory-bound operations) was not investigated. Lesson: benchmark a full epoch, not a few batches.
The residual model did worse	Expected an improvement; got 75.32% against 81.70%.	Kept the result, did not tune on the test set, and listed untested explanations (short schedule, strided convolutions instead of max-pooling). Lesson: a fair negative result beats a manipulated positive one.
ONNX export problems	The legacy exporter is deprecated; the new exporter first printed a version-conversion error and wrote the weights to a separate data file (the .onnx was only 0.04 MB).	Used opset 18, disabled external data and described the dynamic batch axis with dynamic_shapes. Result: one 1.19 MB file that runs for batch sizes 1, 3 and 100.
Misleading generated text	The auto-written conclusion called the ship class (-0.4 pp) the 'largest change', and the README first said 'many' birds were called airplane when the sample showed two.	Reworded to 'most and least favourable change' and corrected the README to what the image shows. Lesson: check generated text against the data.
Leftover code	A static check found unused imports; the tqdm dependency was never used; a few lambda helpers were awkward.	Removed them and re-ran all tests. Lesson: run a linter before the final commit.
Git and GitHub hurdles	No git author was configured; cloning your repo stopped at a sign-in prompt; rewriting commit authors failed with 'Filename too long' on the long path.	Set an author, completed the sign-in in the credential window, and redid the rewrite from a copy at a short path. The push was then verified against GitHub.
Tooling slips	A shell command that wrote many files failed on quoting, and a temporary-scripts folder was not writable.	Switched to the file-writing tool and kept check scripts outside the project. No effect on the delivered code.

7. Likely interview questions and honest answers

Short answers that stay true to what was actually done. If an answer is a hypothesis, it is labelled as one.

Q: Walk me through your project.

See the 2-minute walkthrough in section 2: data, model, training, evaluation, residual comparison, deployment, tests.

Q: Is 81.7% a good accuracy?

It is reasonable for a model with 0.29 million parameters trained for 10 epochs on a CPU. Well-tuned, larger networks trained much longer reach roughly 93-95% on CIFAR-10, so there is clear headroom. The goal here was a clean, reproducible pipeline and a fair comparison, not the highest score.

Q: Why did the ResNet do worse than the baseline?

I did not prove a cause. Candidates: (1) both models are under-trained at 10 epochs and the ResNet learns more slowly (63.7% vs 70.0% validation accuracy at epoch 5); (2) the ResNet downsamples with stride-2 convolutions while the baseline uses max-pooling, which is a second difference besides the skip connections; (3) at about seven convolution layers, with BatchNorm already easing optimisation, skip connections have little to fix. To find out I would run an ablation (max-pooling in the ResNet), a longer schedule, and several seeds.

Q: Is the 6.4 percentage-point gap real or noise?

It is real for this setup. The standard error of an accuracy near 82% on 10,000 test images is about 0.4 pp, so the gap is many standard errors. The caveat is that each model was trained once with one seed, so I have no seed-to-seed variance estimate.

Q: How do you know there is no data leakage?

The split is made by index before any training; normalisation statistics come from the training split only; validation uses its own un-augmented dataset instance; checkpoints and early stopping use validation; the test set is evaluated once at the end. A unit test checks that train and validation indices do not overlap and that validation images are not augmented.

Q: Why is validation accuracy higher than training accuracy?

Training accuracy is averaged over the epoch while the weights are still improving, and it is measured with dropout and data augmentation active. Validation is measured after the epoch, with dropout off and no augmentation. It does not mean validation data is easier.

Q: What is overfitting and is your model overfitting?

Overfitting is when training performance keeps improving while validation performance stalls or worsens. Here validation keeps improving with training and the losses are still falling, so the model is under-trained rather than overfit.

Q: Why BatchNorm, and what does it do at evaluation time?

During training it normalises each channel using the batch mean and variance, which stabilises optimisation and permits a higher learning rate. At evaluation it uses running averages collected during training, which is why the model must be switched to eval mode.

Q: Why spatial dropout, and why those rates?

Neighbouring pixels in a feature map are correlated, so dropping whole channels regularises more effectively than dropping single pixels. 0.1 in the blocks and 0.3 before the classifier are standard starting values; I did not tune them.

Q: Why global average pooling?

It replaces a large flatten-and-dense layer with an average over each feature map, leaving only 1,290 parameters in the head. That reduces overfitting and size.

Q: Why SGD instead of Adam?

SGD with momentum is a strong default for CNN image classifiers and tends to generalise well. AdamW is implemented and selectable in the config, but I did not compare them.

Q: Explain OneCycleLR.

The learning rate rises from a low value (0.004) to a peak (0.1) and then anneals towards zero, updated every batch. The high-LR phase speeds learning and acts as regularisation; the final low-LR phase lets the model settle. It suits short budgets.

Q: How are the parameter counts computed?

Baseline: convolution weights 864 + 9,216 + 18,432 + 36,864 + 73,728 + 147,456 = 286,560; BatchNorm scale and shift 896; linear head 1,290; total 288,746. The ResNet has 308,650: the extra comes from the stem convolution and the 1x1 projection shortcuts.

Q: What is a residual connection and what problem does it solve?

The block outputs F(x) + x instead of F(x). Gradients can flow through the addition directly, which makes very deep networks trainable and lets blocks learn small corrections. When shapes differ, a 1x1 convolution projects x to the right size.

Q: How did you make the comparison fair?

Same seed, data split, preprocessing, optimiser, learning-rate schedule, batch size, epochs, regularisation settings and early-stopping setting, plus comparable parameter counts (288,746 vs 308,650).

Q: What do precision, recall and F1 mean?

Precision is the fraction of predicted positives that are correct; recall is the fraction of real positives that are found; F1 is their harmonic mean. Macro F1 averages F1 over the ten classes (baseline 0.8156, ResNet 0.7518).

Q: How do you read a confusion matrix?

Rows are true classes and columns are predictions; the diagonal is correct predictions and each off-diagonal cell is a specific mistake. Mine shows cat to dog (147) and dog to cat (103) as the biggest errors.

Q: How is the project reproducible?

Seeds for Python, NumPy, PyTorch, CUDA, the split, the shuffle generator and the DataLoader workers; deterministic cuDNN flags; pinned dependencies; one config file. Repeating a quick run gave identical numbers on my CPU. Remaining sources of variation are multi-threaded floating-point order, some non-deterministic CUDA kernels, and resumed runs not restoring the random generator state.

Q: How do you serve the model?

Export to ONNX with a dynamic batch axis, verify against PyTorch (max logit difference 5.7e-06, all 100 predictions identical), and serve with FastAPI: GET /health, and POST /predict, which resizes an uploaded image to 32x32, applies the training normalisation and returns the top three classes with probabilities. The model is loaded once at startup.

Q: What happens if someone uploads a bad file?

A non-image returns HTTP 400, a file over 5 MB returns 413, and a missing file returns 422. These cases are covered by tests.

Q: What would you do with more time or a GPU?

Train longer (for example 30-50 epochs), run three or more seeds, ablate the ResNet downsampling, add stronger augmentation such as Cutout or mixup and label smoothing, try a wider and deeper residual network, and use mixed precision on CUDA - the only path I could not test.

Q: What would you do differently?

Benchmark a full epoch before estimating training time, set up a short virtual-environment path first on Windows, and plan seed repeats from the start so the comparison has an error bar.

Q: What are the limitations?

One run per model, a short 10-epoch schedule, no hyperparameter search, a CPU-only environment, the CUDA mixed-precision path untested, and no explanation of the ResNet result verified by experiment.

Q: What testing did you do?

57 pytest tests that run in about 9 seconds without downloading data: model shapes and layers, residual blocks with and without downsampling, data split and normalisation, seeding, checkpoint round trip, training-step behaviour, early stopping, config parsing, API endpoints, and the comparison-report wording.

8. Limitations and next steps

What the results do and do not show

- Each model was trained once (seed 42), so there is no error bar across seeds.
- Ten epochs is a short schedule; both models were still improving.
- Hyperparameters were set once and not searched. The test set was used a single time per model.
- The reason the ResNet scored lower was not tested; three candidate explanations are listed above.
- Training times are wall-clock on a shared CPU and varied between epochs.
- Mixed precision was not run on CUDA; only the CPU-testable parts were exercised.

Concrete next experiments

- Ablation: swap stride-2 convolutions for max-pooling in the ResNet to separate the effect of skip connections from the downsampling change.
- Train both models for 30-50 epochs with three seeds and report mean and standard deviation.
- Add Cutout or mixup and label smoothing; try a deeper residual network with 64-128-256 channels on a GPU.
- Run the CUDA mixed-precision path and compare speed and accuracy.

9. Appendix

Where things are in the code

File	What it does
configs/config.yaml	Every hyperparameter and path; any value can be overridden on the command line.
src/config.py	Loads YAML into a validated dataclass; command-line overrides; short flag aliases.
src/data.py	Datasets, transforms, seeded split, DataLoaders, denormalise, sample grid, channel statistics.
src/models.py	ConvBlock, BaselineCNN, ResidualBlock, ResNetCNN, build_model, count_parameters.
src/engine.py	Train and evaluate loops, optimiser and scheduler factories, checkpoints, early stopping, AMP hook.
src/visualize.py	Curves, confusion matrix, misclassified grid, comparison plot.
src/utils.py	Seeding, device selection, logging, AverageMeter, Timer.
train.py / evaluate.py / compare.py	Command-line entry points for training, test evaluation, and the comparison report.
export_onnx.py / app/main.py	ONNX export with verification; FastAPI service.
tests/	57 unit tests (models, data, utils and engine, API, reporting).
outputs/	plots, results (metrics and history JSON, comparison.md), logs, model.onnx; checkpoints are git-ignored.

Commands

```
pip install -r requirements.txt
python train.py --model baseline_cnn
python train.py --model resnet_cnn
python evaluate.py --model baseline_cnn
python evaluate.py --model resnet_cnn
python compare.py
python export_onnx.py
uvicorn app.main:app --host 127.0.0.1 --port 8000
pytest -v
```

```
python train.py --quick (2-epoch smoke test)
```

Glossary

Term	Meaning
Epoch	One full pass over the 45,000 training images (351 batches of 128).
Logit	The raw, unnormalised score the model outputs for each class.
Softmax	Turns logits into probabilities that sum to 1.
Receptive field	The region of the input image that influences one activation.
Validation set	Held-out training images used to choose checkpoints; never trained on.
Test set	The official 10,000 images used once for the final score.
pp (percentage points)	The difference between two percentages, e.g. 81.70% minus 75.32% is 6.38 pp.
Standard error	Typical sampling error of an accuracy estimate; about 0.4 pp here.
ONNX	A framework-independent model format run by onnxruntime.

Environment: Python 3.13, PyTorch 2.14.1 (CPU), torchvision 0.29.1, NumPy 2.5.4, scikit-learn 1.9.1, matplotlib 3.11.2, ONNX 1.23.2, onnxruntime 1.31.0, FastAPI 0.162.0, pytest 9.1.1. Hardware: Snapdragon X (X126100, 8 cores), 15.6 GB RAM, Windows 11 ARM64.